

L06: IIoT Time Series Forecasting Lab

ITAI 3377 - IoT and Edge Computing

Professor Ayyagari

Houston City College

Monica Joya

Spring 2026

1. Data Preparation and Preprocessing

The IoT temperature dataset was downloaded from Kaggle and loaded from *data/IOT-temp.csv* using pandas. The raw file contained 97,606 rows across multiple sensor readings, with no missing values in the original columns. The timestamp column (*noted_date*) was converted to a proper datetime format so the data could be sorted in time order.

To make the forecasting task cleaner and easier to reproduce, the data was aggregated into a daily average temperature series. After daily resampling, there were 134 daily observations. This resampling introduced 48 missing daily periods, so a simple time-based interpolation with forward fill and backward fill was used to complete the series. Outliers were checked with the IQR rule, and no daily outliers were flagged.

The data was then split chronologically, with training data from 2018-07-28 to 2018-11-12 and test data from 2018-11-13 to 2018-12-08. A random split was not used because random splitting would break the temporal order and allow future information to leak into the training set.

Normalization consideration: The temperature values in this dataset stay within a narrow, consistent range, and the forecasting models used here (linear regression and random forest) do not strictly require normalized inputs. Normalization is applied later in the VAE section, where the neural network benefits from scaled inputs. For the forecasting pipeline itself, the raw temperature scale is preserved so the predictions remain directly interpretable in real-world units.

2. Model Selection and Training

The forecasting model was built with Nixtla AutoMLForecast (Option 1 from the lab instructions). AutoML was chosen because it keeps the workflow simple, reduces manual tuning, and helps compare candidate models efficiently. This is useful in a lab setting because it saves time while still allowing the model selection process to be data-driven rather than arbitrary.

Two model families were evaluated inside AutoMLForecast: linear regression and random forest. Nixtla handled the forecasting feature setup and model search over a small configuration space that included different lag combinations (1-day, 7-day, and 14-day), a rolling mean transform, and a day-of-week date feature. The search was guided by an Optuna-based trial system that minimized MAE across three rolling-origin windows.

In the baseline run, the best-performing model was LinearRegression. The full baseline test metrics are reported in Section 4.

3. Feature Engineering

Nixtla's AutoMLForecast automatically creates forecasting features from lagged values and time-based information. In addition to those automated features, this notebook includes exactly two custom features:

- **Rolling mean of temperature (rolling_mean_7):** A 7-day rolling average that smooths short-term noise from sensor readings. In IIoT environments, temperature sensors often show short spikes from things like sensor warm-up, temporary environmental changes, or brief interference. A weekly rolling mean captures the underlying temperature trend while filtering out that noise, making the forecast more stable.
- **Day of week (day_of_week):** This feature captures weekly cyclical patterns that commonly appear in IIoT temperature data. In many industrial and building settings, weekday and weekend operations differ significantly. HVAC systems, machinery schedules, and occupancy levels all shift on weekends, which directly affects temperatures near equipment and inside buildings. Including the day index helps the model learn these recurring operational cycles.

These features are useful because IIoT time-series data often contains both short-term sensor noise and repeating patterns tied to human schedules. The rolling mean addresses the first, and the day-of-week feature addresses the second.

4. Model Evaluation and Cross-Validation

Model performance was evaluated with three metrics: MAE (Mean Absolute Error) as the primary metric, MSE (Mean Squared Error) as a secondary metric that penalizes larger errors more heavily, and MASE (Mean Absolute Scaled Error) which compares the model against a naive one-step-ahead baseline. A MASE below 1.0 means the model outperforms the naive forecast.

Baseline Test Results

Metric	Test Set	Cross-Validation
MAE	2.1986	1.7167
MSE	8.3197	4.2151
MASE	2.2018	1.7192

Rolling-origin cross-validation was performed on the training data using three windows, each with a 7-day forecast horizon. This time-aware validation method ensures that the model is always trained on past data and tested on future data, which prevents information leakage and provides a more realistic estimate of how the model would perform in a real deployment. The cross-validation results confirmed that the model performed consistently across multiple historical forecast windows.

Potential Improvements

- **Additional model families:** Expanding the AutoML search to include gradient-boosted trees (like LightGBM or XGBoost) could improve accuracy, since tree-based models often handle nonlinear temperature patterns well.
- **Longer lag windows:** Testing lags beyond 14 days, such as 21 or 30 days, may help the model capture longer-term seasonal trends in the temperature data.
- **External features:** Incorporating weather forecasts, humidity readings, or operational schedules from the IIoT environment could provide the model with additional context that a single temperature series cannot offer.
- **VAE architecture:** Increasing the latent dimension or training for more epochs with a larger dataset could produce more realistic synthetic data and lead to a stronger augmentation effect.

5. Generative Modeling

A simple Variational Autoencoder (VAE) was built with TensorFlow and Keras and trained on sliding windows of the daily temperature series. The VAE architecture consisted of a small encoder (one hidden layer of 16 units) that compresses each 14-day temperature window into a 2-dimensional latent space, and a decoder that reconstructs temperature windows from latent vectors. The model was trained for 20 epochs with a combined reconstruction and KL-divergence loss.

After training, 2 latent samples were drawn from a standard normal distribution and decoded into synthetic temperature windows. The 28 resulting synthetic points were clipped to the training data range and prepended before the original training history to create an augmented training set.

The forecasting workflow was then repeated with the augmented dataset using the same AutoML setup. Both MAE and MSE improved after augmentation, suggesting that the synthetic data gave the forecasting model additional pattern information and helped reduce prediction error. The improvement was modest, which is expected given the small number of synthetic points, but it demonstrated that VAE-based augmentation can support the forecasting task without adding significant pipeline complexity.

6. Individual Reflection

This lab showed how a full time-series workflow can be built in a structured and reproducible way. The most important part was keeping the process chronological at every step, from splitting the data to running cross-validation, so that the evaluation stayed honest and no future data leaked into training.

Using Nixtla AutoMLForecast made the forecasting stage easier to manage, since I did not have to manually search through model configurations. The automated search with Optuna trials handled that while still being transparent about what was selected. The

VAE section added a useful generative modeling component without making the notebook too complicated.

One useful lesson from this lab is that even small feature choices, such as a rolling mean and a time-based feature, can help forecasting models capture useful patterns when they are grounded in domain reasoning. Another lesson is that synthetic data does not always create a large gain, but it can still improve results slightly when it adds realistic variation. Overall, the workflow met the assignment goal by combining data preparation, forecasting, evaluation, augmentation, and written interpretation in one pipeline.

7. References

- Muramatsu, K. (n.d.). *IoT temperature forecasting* [Dataset and notebook]. Kaggle. <https://www.kaggle.com/code/koheimuramatsu/iot-temperature-forecasting>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://pandas.pydata.org/>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://scikit-learn.org/>
- Nixtla. (2024). *MLForecast: Scalable machine learning for time series* [Software]. <https://github.com/Nixtla/mlforecast>
- Abadi, M., et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software]. <https://www.tensorflow.org/>
- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*.